# Tuning Hashes

Selected Projects in Performance Engineering of
Hash-Based Data Structures and Hash Functions

Calvin James Higgins
Department of Computer Science and Statistics
University of Rhode Island
March 19, 2026

A portfolio presented for the degree of
Master of Science in Computer Science

# 1 Introduction

I extract and tune parameters to improve performance. I selected three projects to illustrate this single theme .

TODO: Additionally, the student will add a single overall introduction (300-words or more) that motivates why they selected those three projects as exemplary of their accomplishments in the program.

# 2 Non-Uniform Binary Fuse Filters

This project was completed for CSC 592: Algorithms for Big Data.

## 2.1 Motivation

Distributed systems often require fast and small set data structures. For example, content delivery networks avoid caching one-hit wonders by querying a visited set [6]. Supporting such exact membership queries on a large universe[1] $U$ requires $\Omega(\lg |U|)$ bits per key. For large enough sets, this is impractical. By permitting false positive results with some probability $\epsilon$, the lower bound is reduced to $\lg \frac{1}{\epsilon}$ bits per key [2]. In other words, *approximate* membership queries can be much more memory efficient than exact membership queries [1, 5, 4].

A binary fuse filter [4] is a recent static probabilistic data structure for approximate membership queries. Membership information is stored in an array of $m$-bit values partitioned into disjoint segments. To query the filter, a key is hashed to a fingerprint value and to three array entries located in consecutive segments (see Figure 1). If the bitwise-XOR of the entries is equal to the fingerprint, then the query *might* be in the set. Otherwise, the query *definitely* is not in the set.

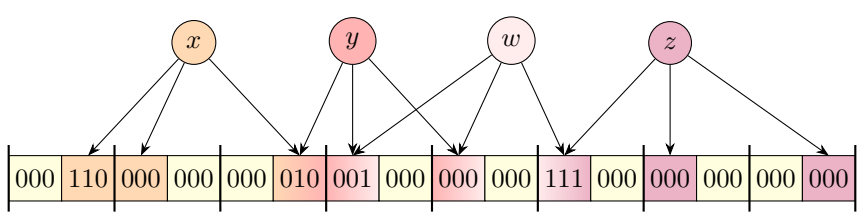


**Figure 1:** *Binary fuse filter with 16-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The filter contains keys* $x$ *,* $y$ *,* $w$ *and* $z$ *, with fingerprints* 100 *,* 011 *,* 110 *and* 111 *, respectively. Each key hashes to three consecutive segments. Corresponding entries are marked by arrows and matching colors. For example, since the filter contains* $x$ *, the bitwise-XOR* $110 \oplus 000 \oplus 010$ *is the fingerprint* 100 *.*

[1] Set of all keys.

To construct a binary fuse filter, choose each array entry such that, for each key, the bitwise-XOR of the three corresponding array entries is equal to the fingerprint. This requires solving a system of linear equations with variables in $\mathbb{F}_2^m$ (i.e. $m$-bit vectors) where addition is performed using bitwise-XOR. Let $h_1$, $h_2$ and $h_3$ be the hash functions, $f$ be the fingerprint function, and $A$ be the array. Each key $k$ yields a linear equation

$$A[h_1(k)] \oplus A[h_2(k)] \oplus A[h_3(k)] = f(k)$$

that must be satisfied (see Figure 2). If some equation is not satisfied, the filter will incorrectly report the corresponding key is *definitely* not in the set. Since approximate membership queries allow only false positives and not false negatives, the choice of array entries must satisfy every equation.

$$A[1] \oplus A[2] \oplus A[5] = 100 \qquad A[5] \oplus A[6] \oplus A[8] = 011$$

$$A[6] \oplus A[8] \oplus A[10] = 110 \qquad A[10] \oplus A[12] \oplus A[15] = 111$$

**Figure 2:** *System of linear equations with variables in $\mathbb{F}_2^3$ induced by Figure 1. Each $A[i]$ is the i-th entry in the backing array $A$ (zero-indexed). The keys are $x$, $y$, $w$ and $z$ with fingerprints 100, 011, 110 and 111, respectively. Array entries and fingerprints are marked with corresponding key color(s). There is one equation for each key.*

To solve the system, model it as a hypergraph. A hypergraph is a graph where edges may contain any subset of vertices. For each entry, create a vertex. For each equation, create an edge containing entries (vertices) in the equation (see Figure 3). Then, 2-peel the hypergraph, that is:

1. Remove vertices of degree zero, then set their entries to $0^m$.
2. Remove vertices of degree one and their incident edges. Then, set their entries to the bitwise-XOR of the fingerprint and the two other entries in the edge. If the other entries are not yet set, defer until they are set.
3. Repeat until there are no vertices of degree less than two, or until the empty hypergraph is reached (i.e. all entries set).

If the empty hypergraph is reached, the hypergraph is called 2-peelable. The system is solved if and only if its hypergraph is 2-peelable.

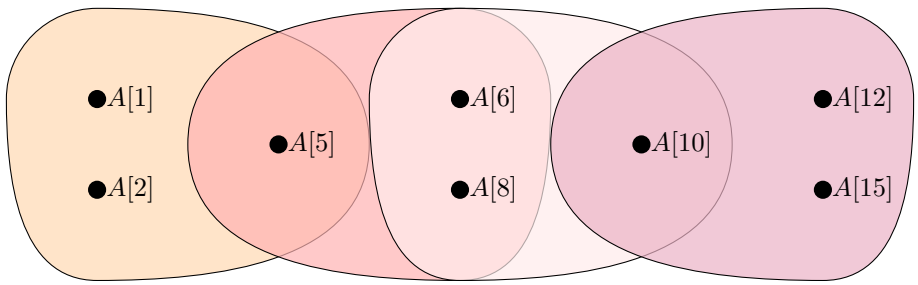


**Figure 3:** *Hypergraph model of Figure 2. Each $A[i]$ is the $i$-th entry in the backing array $A$ (zero-indexed). Hyperedges correspond to keys $x$, $y$, $w$ and $z$, colored accordingly. Vertices of degree zero not shown.*

In the filter setting, edges represent keys and vertices represent array entries. Lower edge size means cheaper queries, since fewer array entries (vertices) are accessed per key (edge). Higher edge density means lower space usage, since there are more keys (edges) per array entry (vertex).

Many hypergraph families exhibit sharp phase transitions in peelability [3, 7]. When the edge density is below a critical value, called the 2-peelability threshold, sampled hypergraphs are almost surely 2-peelable. When the density exceeds this threshold, they are almost surely not. The threshold is therefore a direct measure of space efficiency. A higher threshold means more keys can be stored per array entry.

Binary fuse filters induce the fuse graph family, which achieves the supremum 2-peelability threshold among 3-uniform hypergraph[2] families [7]. This threshold occurs at an edge density of 0.918, or roughly $1.089 \lg \frac{1}{\epsilon}$ bits per key [7]. No family of 3-uniform hypergraphs has higher 2-peelability threshold. More generally, a binary fuse filter with $m \geq 3$ hash functions[3] saturates the 2-peelability threshold for $m$-uniform hypergraphs [7].

However, this optimality result is confined to uniform hypergraphs. There may exist families of non-uniform hypergraphs with higher 2-peelability thresholds. If such families also have a low average edge size (i.e. few hashes and accesses per query), filters based on them could potentially reduce memory overhead without degrading query latency and throughput. Therefore, we ask:

1. How can we systematically construct non-uniform hypergraph families with peelability thresholds comparable to or surpassing those of fuse graph families?

[2] In general, a hypergraph is $m$-uniform if every edge contains exactly $m$ vertices.

[3] Only binary fuse filters with three or four hash functions are practical. With four hash functions, the 2-peelability threshold is already 0.977, yielding about $1.023 \lg \frac{1}{\epsilon}$ bits per key or within 2.3% of optimal [7].

2. How can we design efficient approximate membership query data structures by exploiting peelable non-uniform hypergraph families?

## 2.2 Summary

### 2.2.1 Non-Uniform Hypergraphs

We computed 2-peelability thresholds for 10K non-uniform fuse graph families with mean edge size between 3 and 4 (see Figure 4). As peelability is monotonic in edge density, we used binary search. At each candidate threshold, we sampled three hypergraphs with 1M edges. If the majority were peelable, we considered the family peelable at that threshold. We implemented this algorithm in C++ and found thresholds in parallel on a compute cluster.

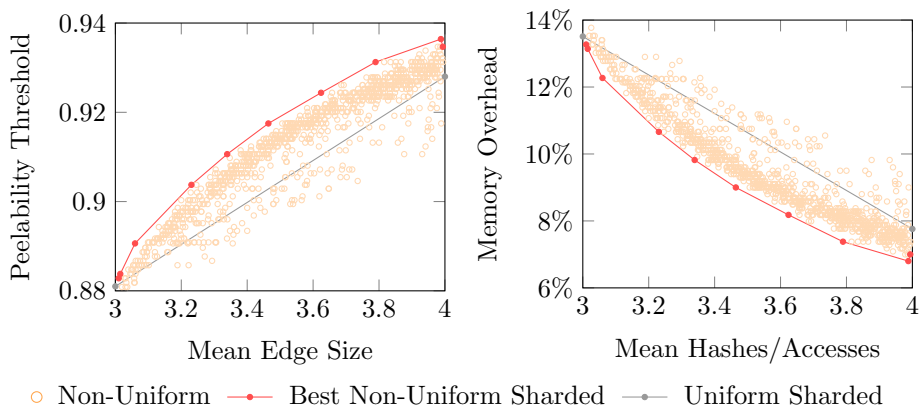


**Figure 4:** *Empirical peelability thresholds and memory overheads for non-uniform fuse graphs with 1M edges and mean edge size between 3 and 4. Orange dots show 1K sampled families. Red shows the best non-uniform sharded families. Gray shows the sharded uniform baseline. Many non-uniform families outperform the uniform baseline. Left: mean edge size vs. peelability threshold (higher is better). Right: theoretical query cost vs. memory overhead (lower is better).*

While $m$-uniform graphs have integer mean edge sizes, they can be extended to any mean edge size via sharding[4] [4]. For example, splitting edges evenly between edge-disjoint 3-uniform and 4-uniform subgraphs yields a mean edge size of 3.5. Many non-uniform families outperform this baseline, and therefore afford smaller filters. Interestingly, some non-uniform families also outperform uniform families at integer mean edge sizes. Since peelability thresholds are defined in the limit of large graphs, a non-uniform family can be more peelable in practice than a uniform family with nearly the same mean edge size. For

[4]Sharding partitions a dataset into disjoint subsets. For hypergraphs, each subset becomes an edge-disjoint subgraph. For filters, each subset is stored in a separate filter.

example, one family achieves $\approx 1\%$ memory savings over the uniform baseline at mean edge size 4.

These data suggest two conjectures. Let $\mathcal{H}_d$ denote the set of fuse graph families with mean edge size $d$ and let $\tau(\mathcal{F})$ denote the 2-peelability threshold of a family $\mathcal{F}$. Define $\Gamma(d) = \sup\{\tau(\mathcal{F}) : \mathcal{F} \in \mathcal{H}_d\}$. We conjecture that $\Gamma(d)$ is continuous, monotonically increasing, and concave. Since a concave $\Gamma$ would lie above any linear interpolation, this implies that sharding is suboptimal. We further conjecture that for $d \notin \mathbb{Z}$, the supremum is attained by the family with a fraction $(1-p)$ of $\lfloor d \rfloor$-edges and a fraction $p$ of $\lceil d \rceil$-edges, where $p = d - \lfloor d \rfloor$. For example, for $d = 3.5$, this family contains 50% 3-edges and 50% 4-edges. Both conjectures remain open.

### 2.2.2 Non-Uniform Binary Fuse Filters

We implemented and optimized a non-uniform binary fuse filter with 50% 3-edges and 50% 4-edges in C++ (i.e. each key hashes to either 3 or 4 array entries). This edge distribution offered the greatest advantage over sharding uniform binary fuse filters. We considered three query algorithms: conditional branch, branchless and phantom entry described below (see Table 1 for metrics and Section A.2 for pseudocode).

| Algorithm | Hashes | Branch Mispredictions | Cache Misses |
|---|---|---|---|
| Conditional Branch | 3.5 | 0.5 | 3.5 |
| Branchless | 4 | 0 | 4 |
| Phantom Entry | 4 | 0 | 3.5 |

**Table 1:** *Metrics for non-uniform binary fuse filter query algorithms. Hashes, branch mispredictions, and cache misses are mean per query.*

In all three algorithms, we report that the key *might* be in the set if the XOR of the accessed array entries equals the fingerprint. The algorithms differ only in how they handle variable edge size when computing the XOR.

In the conditional branch algorithm (see Figure 1), we XOR the first three array entries. If the key is a 4-edge, we XOR in the fourth entry. This branch mispredicts on at least half of all queries.

In the branchless algorithm (see Figure 2), we load all four array entries unconditionally. If the key is a 3-edge, we zero the fourth value with a mask before XORing all four values. For large filters, all four loads will miss the cache.

In the phantom entry algorithm (see Figure 3), we load all four array entries unconditionally. However, if the key is a 3-edge, we zero the fourth hash address

before the load, redirecting it to $A[0]$. To support this, we shift all valid entries left by one index and set $A[0] = 0$ at construction time (see Figure 5). Unlike the branchless algorithm, the fourth load for 3-edges hits the cache, since $A[0]$ is accessed in 50% of queries.

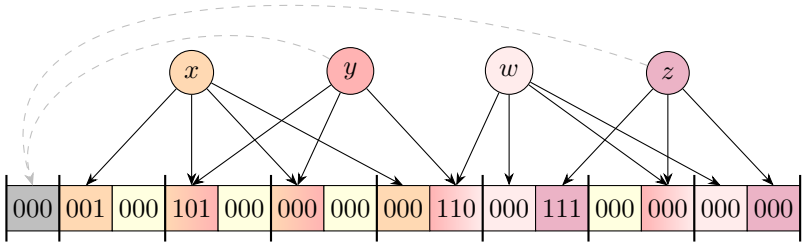


**Figure 5:** *Non-uniform binary fuse filter with phantom entry, 14-entry array, 2-entry segments, and 3-bit fingerprints. Vertical bars mark segment boundaries. The phantom entry is colored gray. The filter contains keys* $x$, $y$, $w$ *and* $z$, *with fingerprints* 100, 011, 110 *and* 111, *respectively. Each key hashes to three or four consecutive segments. Corresponding entries are marked by arrows and matching colors. Dashed gray arrows indicate hashes to the phantom entry. For example, since the filter contains* $y$, *the bitwise-XOR* $101 \oplus 000 \oplus 110 \oplus 000$ *is the fingerprint* 011. *See Figure 6 and Figure 7 for the induced system of equations and hypergraph.*

Surprisingly, only the conditional branch algorithm achieved higher query throughput than the 4-uniform binary fuse filter, and even this fell short of an acceptable time-space tradeoff. Since we expected nearly all cycles to be spent stalled on memory loads, reducing cache misses per query should have improved throughput proportionally. In practice, however, only 80% of cycles were stalled on loads, and there were no other stalls.

We hypothesize that out-of-order (OoO) execution stalls before it can compute load addresses for the next key. As a result, when outstanding loads complete, hashes for the next query have not yet been computed, and the subsequent loads are delayed by the full hash latency. This dependency also explains the conditional branch algorithm's advantage: its lower hash latency reduces this delay.

The root cause is likely one of two structural hazards: either a shortage of miss status holding registers (MSHRs) prevents prefetching the next key (and therefore precomputing its hashes), or the reorder buffer (ROB) is too small to precompute enough hashes for future keys. Based on microarchitectural parameters and the benchmark assembly, the first explanation is more likely.

#### 2.2.3 Future Work

We see four promising directions for future work. First, batching may yield a non-uniform binary fuse filter with a competitive time-space tradeoff. Second, interleaving the segments of a binary fuse filter could improve cache locality during queries. Third, introducing additional weak points into the fuse graph would enable greater parallelism during filter construction. Currently, the fuse graph has only two weak points, one at each end, limiting peeling to two threads. The recommended workaround is sharding, but this may sacrifice memory efficiency. Fourth, we aim to prove the conjectures stated in Section 2.2.1, for which the methods of [3] and [7] may be useful.

### 2.3 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.

## 3 Tuning Secure Learned Bloom Filters

This project was completed for CSC 592: Machine Learning Security.

### 3.1 Abstract

TODO: A 400–500-word summary of the technical contributions of the work, what was accomplished overall, and the limitations of their work.

### 3.2 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.

## 4 Entropy-Aware Hashing

This project was completed for CSC 519: Computer Networks.

### 4.1 Abstract

TODO: A 400–500-word summary of the technical contributions of the work, what was accomplished overall, and the limitations of their work.

### 4.2 Reflection

TODO: A 250-word self reflection of what skills were developed or enhanced in completing those projects.

# References


[1] Burton H. Bloom. "Space/Time Trade-Offs in Hash Coding with Allowable Errors". In: *Commun. ACM* (1970). URL: https://doi.org/10.1145/362686.362692.

[2] Larry Carter et al. "Exact and Approximate Membership Testers". In: *Symposium on Theory of Computing (STOC)*. 1978. URL: https://doi.org/10.1145/800133.804332.

[3] Martin Dietzfelbinger and Stefan Walzer. "Dense Peelable Random Uniform Hypergraphs". In: *European Symposium on Algorithms (ESA)*. 2019. URL: https://doi.org/10.4230/LIPIcs.ESA.2019.38.

[4] Thomas Mueller Graf and Daniel Lemire. "Binary Fuse Filters: Fast and Smaller Than Xor Filters". In: *Journal of Experimental Algorithmics (JEA)* (2022). URL: http://dx.doi.org/10.1145/3510449.

[5] Thomas Mueller Graf and Daniel Lemire. "Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters". In: *Journal of Experimental Algorithmics (JEA)* (2020). URL: http://dx.doi.org/10.1145/3376122.

[6] Bruce M. Maggs and Ramesh K. Sitaraman. "Algorithmic Nuggets in Content Delivery". In: *SIGCOMM Comput. Commun. Rev.* (2015). URL: https://doi.org/10.1145/2805789.2805800.

[7] Stefan Walzer. "Peeling Close to the Orientability Threshold: Spatial Coupling in Hashing-Based Data Structures". In: *Symposium on Discrete Algorithms (SODA)*. 2021. URL: https://doi.org/10.1145/3711822.

# A Non-Uniform Binary Fuse Filters

## A.1 Non-Uniform Binary Fuse Filter Alternative Views

$$A[1] \oplus A[3] \oplus A[5] \oplus A[7] = 100$$

$$A[3] \oplus A[5] \oplus A[8] \oplus A[0] = 011$$

$$A[8] \oplus A[9] \oplus A[13] \oplus A[14] = 110$$

$$A[10] \oplus A[13] \oplus A[15] \oplus A[0] = 111$$

**Figure 6:** *System of linear equations with variables in $\mathbb{F}_2^3$ induced by Figure 5. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). The keys are $x$, $y$, $w$ and $z$ with fingerprints 100, 011, 110 and 111, respectively. Array entries and fingerprints are marked with corresponding key color(s). The phantom entry is colored gray. There is one equation for each key.*

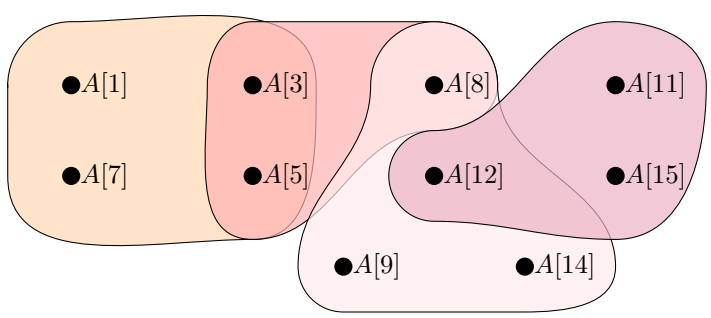


**Figure 7:** *Hypergraph model of Figure 6. Each $A[i]$ is the i-th entry in the backing array A (zero-indexed). Hyperedges correspond to keys $x$, $y$, $w$ and $z$, colored accordingly. Vertices of degree zero and phantom entry $A[0]$ not shown.*

### A.2 Non-Uniform Binary Fuse Filter Query Algorithms

**Algorithm 1** Conditional branch algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k)] ⊕ A[Hash2(k)] ⊕ A[Hash3(k)]
    if IsFourEdge(k) then
        f′ ← f′ ⊕ A[Hash4(k)]
    end if
    return f == f′
end function
```

**Algorithm 2** Branchless algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k)] ⊕ A[Hash2(k)] ⊕ A[Hash3(k)]
    f′ ← f′ ⊕ (-IsFourEdge(k) & A[Hash4(k)])
    return f == f′
end function
```

**Algorithm 3** Phantom entry algorithm.

```
function Contains(k)
    f ← Fingerprint(k)
    f′ ← A[Hash1(k) + 1] ⊕ A[Hash2(k) + 1] ⊕ A[Hash3(k) + 1]
    f′ ← f′ ⊕ A[-IsFourEdge(k) & (Hash4(k) + 1)]
    return f == f′
end function
```

# B Tuning Secure Learned Bloom Filters

# C Entropy-Aware Hashing